

0.83.2. Un son quand on ramasse : note — et un bruit contre le mur

Le labyrinthe du 0.82.1, plus un bruit quand on se cogne : `bruit(4, 8)`, sur la voix du bruit.

1. Ce qu'il faut comprendre

C'est le labyrinthe du 0.82.1, plus un son quand le A se **cogne** contre un mur.

Ce qui est nouveau ici : `bruit(duree, volume)`. Elle frappe sur la **voix du bruit** (la 4e) : pas une note, un « toc ».
`bruit(4, 8)` : 4 images, volume 8.

Où le mettre : dans le `else` du test du mur. Si la case d'arrivée **n'est pas** un M, on avance ; **sinon** (`else`), c'en est un : on reste, et on fait « toc ».

Et une note quand on ramasse la pièce, comme au 0.83.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <bruit>` → [0.83.1. La fonction bruit\(\) — un bruit](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1361 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	78 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
main()	441 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres

CopierTuiles	85 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	288 o	← CopierTuiles	– 18 lettres sur 43
Notes	120 o	← note()	– la table des notes de musique
"MMMMMMMMMMMMMM..."	20 o	← texte()	– un texte gravé
"MMMMMMMMMMMMMM..."	20 o	← texte()	– un texte gravé
"SCORE"	5 o	← texte()	– un texte gravé

Lettres gravées : ACEMOPRS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

poser()	× 6	68 o sur place, dans main()	+ AdresseCase (10 o)
note()	× 1	56 o sur place, dans main()	+ Notes (120 o)
bruit()	× 1	39 o sur place, dans main()	
texte()	× 3	33 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 4	20 o sur place, dans main()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
nombre()	× 1	10 o sur place, dans main()	+ EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : les mêmes qu'au 0.82.1 ----

#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
#include <bruit>      // joue un bruit
#include <note>       // joue une note
#include <nombre>     // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 3;
```

3. UNE VARIABLE — Y

```
uint8_t y = 5;
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE LABYRINTHE : tout en lettres M ----
    //
    // ligne 2 : M M M M M M M M M M M M M M M M M le haut
    // ligne 3 : M M M M M M M M M M M M M M M M M
    // ligne 5 : M A M P M A : départ, P : la pièce
    // ... M M M M M M M M M M M M M M M M M
    // ligne 10 : M M M M M M M M M M M M M M M M M le mur du milieu s'arrête ici...
    // ligne 11 : M M M M M M M M M M M M M M M M M ...les lignes 11 à 13
    // ligne 13 : M M M M M M M M M M M M M M M M M sont libres : on passe
    // ligne 14 : M M M M M M M M M M M M M M M M M le bas

    // Le haut et le bas : une ligne de 20 M, en UN seul texte.
    texte(0, 2, "MMMMMMMMMMMMMMMMMMMMMMMM");
    texte(0, 14, "MMMMMMMMMMMMMMMMMMMMMMMM");

    // Les deux côtés : un M par ligne, à gauche (colonne 0) et à droite (colonne 19).
    for (uint8_t l = 3; l < 14; l++) {
        poser(0, l, ALPHABET[12]);
        poser(19, l, ALPHABET[12]);
    }

    // Le mur du milieu, en colonne 10, des lignes 3 à 10 seulement : ouvert en bas.
    for (uint8_t l = 3; l <= 10; l++) {
        poser(10, l, ALPHABET[12]);
    }

    poser(16, 5, ALPHABET[15]); // la pièce P, de l'autre côté du mur
    poser(x, y, ALPHABET[0]); // le A, à son départ
    texte(0, 17, "SCORE");

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();
        if (chaque(250)) { // au plus un pas tous les 250 ms (le 0.77)

            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // | |
            // | | +-- le M, la lettre des murs
            // | +----- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) { // une flèche est tenue (|| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1); // efface le A à l'ancienne case
                    x = nx; // la case d'arrivée devient
                    y = ny; // la case du A
                }
            }
        }
    }
}
```

```

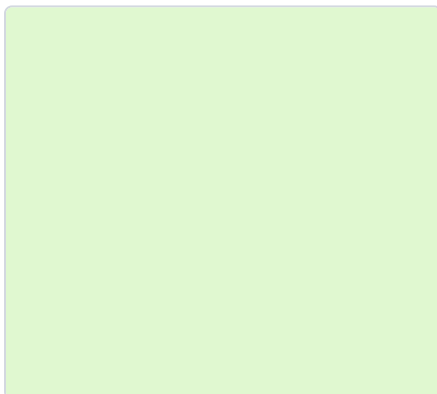
    poser(x, y, ALPHABET[0]);    // pose le A à la nouvelle case
} else {
    // ÉTAPE 3 bis : c'EST un mur. Le A ne bouge pas, et on fait « toc ».
    //
    //   bruit(4, 8);
    //       | |
    //       | +--- volume : 8 (de 0 à 15)
    //       +----- durée : 4 images, un bruit très court
    //
    // bruit joue sur la VOIX DU BRUIT, la 4e : pas une note, un choc.
    bruit(4, 8);
}
}
}

// La pièce : une seule fois (score == 0), avec un « ding » (le 0.83).
if (x == 16 && y == 5 && score == 0) {
    score = 1;
    note(1, D05, 10, 12);
}
nombre(6, 17, score);
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



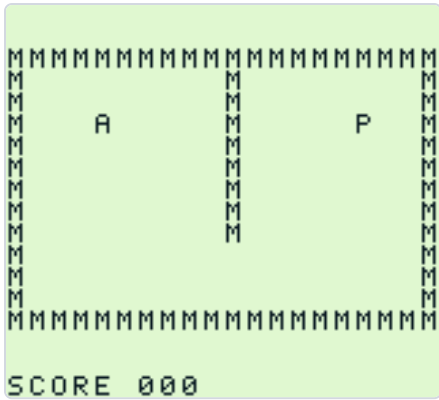
la toute première image — la console vient de démarrer — image 1



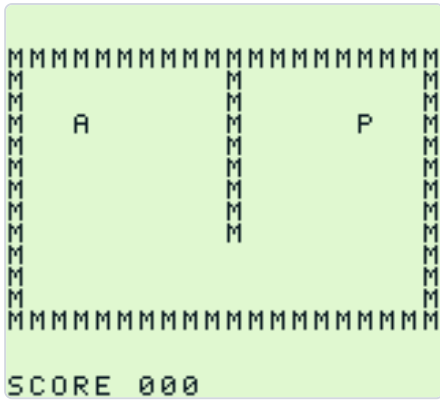
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



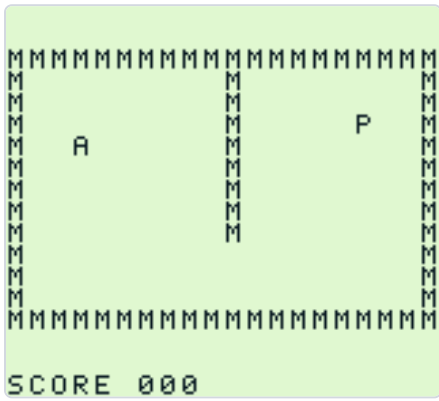
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le labyrinthe ; contre un mur, un « toc » ; sur la pièce, un « ding ».

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ contre le mur, la voix du bruit a frappé
- ✓ le A est resté devant le mur